GCF-FER: Hybrid CNN–Graph Convolutional Features for Facial Expression Recognition on JAFFE

A PyTorch pipeline that benchmarks a **hybrid CNN + Graph Convolutional Network (GCN)** architecture — the *Graph Convolutional Features (GCF)* approach — for **seven-class facial expression recognition (FER)** on the JAFFE dataset. The notebook trains and evaluates nine ImageNet-pretrained backbones under a unified protocol, visualises the results, and compares them with the figures published in the original GCF paper (Kassab et al., 2024, arXiv:2407.02361).

---

1. Overview

Facial expressions are one of the most direct non-verbal channels of human emotion. Automatic FER maps a face image to an emotion category (angry, disgust, fear, happy, neutral, sad, surprise). Conventional CNN classifiers learn strong appearance features but treat the face as a flat feature vector. The GCF idea augments CNN features with a **graph-based branch**, so that relational structure between feature groups can be modelled in addition to raw appearance.

This project:

1. Prepares and organises the JAFFE dataset into class-labelled folders.
2. Builds a configurable model (`cnn`, `gcn`, or `cnn+gcn`) on top of a pretrained CNN backbone.
3. Trains and evaluates **nine backbones** (VGG, ResNet, DenseNet, EfficientNet families) on the same split.
4. Produces learning curves, per-backbone accuracy charts, a confusion matrix, and a comparison against published results.

2. Motivation and Applications

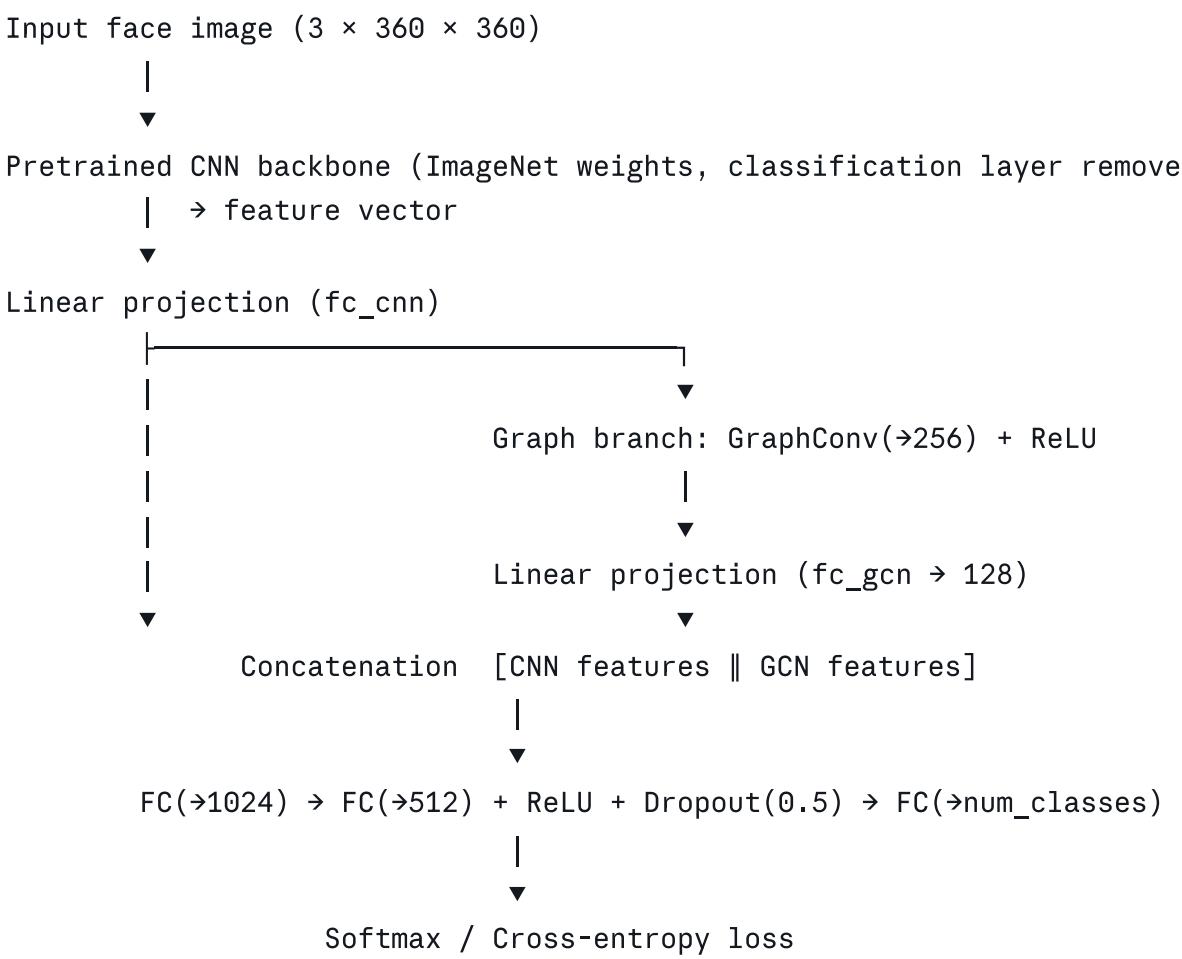
Reliable FER is a building block for affective computing, including:

- **Human–computer interaction** — interfaces that adapt to user emotion.
- **Education and e-learning** — estimating learner engagement and confusion.
- **Healthcare and well-being** — supporting emotion monitoring in clinical and assistive settings.
- **Driver and operator monitoring** — detecting stress or fatigue cues.
- **User-experience and market research** — measuring reactions to products and content.

A small, well-controlled dataset such as JAFFE is a practical testbed for validating a new architecture before scaling to large in-the-wild datasets (e.g., RAF-DB, FER+).

3. Method

3.1 Pipeline



3.2 Model modes

Mode	Description
`cnn`	Backbone features only (baseline).
`gcn`	Graph-branch features only.
`cnn+gcn`	Concatenation of both branches (full GCF model).

3.3 Supported backbones

`vgg16`, `vgg19`, `resnet18`, `resnet34`, `resnet50`, `densenet121`, `densenet169`, `densenet201`, `efficientnet_b0` (all evaluated), plus `inception_v3` (supported by the model class, not evaluated here).

3.4 Training configuration

Parameter	Value
Framework	PyTorch, torchvision, PyTorch Geometric
Initialisation	ImageNet-pretrained backbone, fine-tuned end to end
Optimiser	Adam (learning rate 5×10^{-5}, weight decay 1×10^{-4})
Loss	Cross-entropy
Epochs	20
Batch size	18
Regularisation	Dropout (p = 0.5) in the classification head
Input size	360 × 360 RGB
Split	70 % train / 10 % validation / 20 % test
Hardware	Google Colab, NVIDIA Tesla T4 GPU

4. Dataset

JAFFE (Japanese Female Facial Expression; Lyons et al., 1998) contains **213 grayscale images** of 10 Japanese female subjects posing **seven expressions**: angry, disgust, fear, happy, neutral, sad, and surprise (roughly 29–32 images per class).

The raw dataset is a flat folder whose filenames encode the expression (`AN`, `DI`, `FE`, `HA`, `NE`, `SA`, `SU`). The notebook parses these codes and copies the images into per-class sub-folders (`jaffe_sorted/<class>/`), which is the layout expected by `Data_Loader.py`.

> The dataset is not redistributed in this repository. Obtain it from its original source and respect its licence terms (research use).

5. Results

Single run, `cnn+gcn` mode, test-set accuracy:

Backbone	Test accuracy
VGG19	**91.67 %**
ResNet34	86.11 %
DenseNet201	86.11 %
VGG16	83.33 %
ResNet18	83.33 %
EfficientNet-B0	83.33 %
ResNet50	80.56 %
DenseNet121	69.44 %
DenseNet169	69.44 %

Generated artefacts (saved to Google Drive): `model_results.xlsx`, `vgg16_curves.png` (loss/accuracy curves), `accuracy_per_model.png`, `vs_paper_comparison.png` (reproduction vs. published Table I), and `confusion_matrix.png`.

How to read these numbers. The reported accuracies are multiples of 1/36, which indicates a test set of only 36 images — a single image changes accuracy by about 2.8 percentage points. Results come from one run per backbone without a fixed random seed, so differences of a few points between backbones are within noise and should not be over-interpreted. Several backbones fall below the figures reported in the paper; given the tiny evaluation set and single-run protocol, this is a reproduction gap to be investigated rather than a definitive ranking.

The CNN-only vs. CNN+GCN ablation cell was interrupted before completion in the recorded run, so no complete ablation is reported here.

6. Implementation Notes and Known Limitations

For transparency and reproducibility, the following points about the code as written should be kept in mind:

1. **Graph definition.** `GNNLayer` uses a fixed `edge_index` over node indices 0–8. In the current definition the source and target rows are identical, so every edge is a **self-loop** and no information is exchanged between different nodes. In addition, the graph layer receives a `(batch_size × features)` matrix, so node indices refer to *samples in the mini-batch*, not to facial regions within one image. As a result, the graph branch behaves like an additional learned linear transform rather than genuine message passing over a per-image region graph. A faithful implementation of the paper's idea

would build one graph per image (e.g., nine region nodes with the intended adjacency) and aggregate over that graph.

2. **Class indexing.** Labels are mapped to 1–7 while `num_classes = 8`, leaving index 0 unused.
3. **Checkpoint naming.** The saved-weights filename does not include the backbone name, so successive backbones overwrite each other when trained in a loop.
4. **Evaluation protocol.** The test accuracy is taken from the final epoch (no best-checkpoint selection), on a single random split.
5. **Subject overlap.** JAFFE contains multiple images per subject; a random image-level split may place the same subject in both training and test sets. A subject-independent split is recommended for stricter evaluation.

7. Repository Structure

File	Purpose
`GCF.ipynb`	End-to-end notebook: data preparation, model definition, training, evaluation, plots.
`Data_Loader.py`	Loading, preprocessing, splitting, and batching of the dataset (uploaded to Colab at runtime).
`Training.py`	Training and validation routine (`train`).
`GCF_Model.py` / notebook cell	`GCF` and `GNNLayer` model definitions.
`requirements.txt`	Python dependencies.

8. Requirements

- Python 3.x
- PyTorch and torchvision
- PyTorch Geometric
- scikit-learn, pandas, NumPy, OpenCV, Pillow
- matplotlib, seaborn, openpyxl (plots and Excel export)
- A CUDA-capable GPU is recommended (the notebook falls back to CPU)

9. Installation

```bash
```

```
git clone https://github.com/4qlaa7/GCF.git
cd GCF
pip install -r requirements.txt
pip install torch_geometric
```

10. Usage (Google Colab)

1. Upload the JAFFE images to Google Drive (e.g., `MyDrive/GRC/jaffe`).
2. Open `GCF.ipynb` in Colab and select a **GPU runtime**.
3. Run the first cells to install dependencies and mount Drive.
4. Run the sorting cell to create `jaffe_sorted/<class>/` folders.
5. Upload `Data_Loader.py` and `Training.py` when prompted.
6. Run the data-loading, model-definition, and training cells; results and figures are saved to the Drive folder.

Key settings to adjust:

```python
base_model = 'vgg16'     # any supported backbone
mode       = 'cnn+gcn'   # 'cnn' | 'gcn' | 'cnn+gcn'
num_epochs = 20
batch_size = 18
```

11. Future Work

- Replace the fixed batch-level graph with a **per-image region graph** and validate it against the CNN-only baseline.
- Run repeated experiments with fixed seeds and **k-fold, subject-independent** cross-validation, reporting mean ± standard deviation.
- Complete the CNN vs. CNN+GCN ablation across all backbones.
- Scale to larger in-the-wild datasets (RAF-DB, FER+) and add data augmentation.

12. References

- Kassab et al., Graph Convolutional Features (GCF) model, 2024. arXiv:2407.02361.
- Lyons, M., Akamatsu, S., Kamachi, M., Gyoba, J., *Coding facial expressions with Gabor wavelets*, 1998 (JAFFE dataset).

13. Contributions

Contributions are welcome — please fork the repository, open an issue, or submit a pull request.

14. License

Distributed under the MIT License. See `LICENSE` for more information.